

IAM Hardening Summary - Northstar Assist

Tarie Nosworthy

18 September 2026

Contents

Northstar Assist — Least-Privilege Access Controls	1
1. Method	1
2. Roles reviewed	2
A finding about where the permissions actually live	2
3. Changes applied	3
3.1 Harness execution role	3
3.2 Gateway service role — 2 wildcards to 0	4
3.3 Knowledge base service role	4
4. Surviving wildcards — individual justification	5
5. Verification — and why it was built in	5
6. What an attacker gains — before versus after	6
7. Remaining recommendations	6
Evidence	7

Northstar Assist — Least-Privilege Access Controls

Assessment date: 18 September 2026 **Author:** Tarie Nosworthy **Environment:** AWS account 118924230273, us-east-1

Every policy document quoted here was **read from the live account**, narrowed programmatically, applied, and verified. Nothing was retyped. The complete before/after documents and unified diffs are in `submission/iam/`, the machine-readable change log in `evidence/iam/iam_changes.json`, and the generator's own output in `evidence/iam/generated-change-log.md`.

1. Method

The policies were narrowed, not replaced.

The obvious approach — write ideal policies from scratch and overwrite the console's — has two problems. It requires inventing IAM action names, some of which do not exist. And it produces no honest before/after comparison, because the “before” becomes a description rather than a document.

Instead `scripts/20_harden_iam.py` reads what the console actually attached and applies these transformations:

Transformation	Rule
NARROWED	A statement granting an action on a broad resource is rewritten to the specific ARNs this system uses, resolved from live state
SPLIT	A statement whose actions span several resource groups is divided so each carries its own scope
REMOVED	A statement whose actions serve only a feature that was never configured is deleted entirely
ACTION-REMOVED	An individually indefensible action is stripped from an otherwise legitimate statement
UNCHANGED-WILDCARD	Flagged for explicit justification — never silently accepted

Two design details that mattered in practice:

Feature detection is by predicate, not by an action list. The console grants `GetBrowserSession`, `UpdateBrowserStream`, `s3files:ClientMount` and similar. A hardcoded list drifts out of date the moment AWS adds an action, and the first attempt here matched none of the real names. The matchers key on each feature’s vocabulary instead.

The model ARNs come from the inference profile itself. A cross-Region profile requires both the profile ARN *and* the regional foundation-model ARNs it can route to — granting only the profile produces `AccessDenied` at invoke time. Rather than assume which regions, the script reads them from the live profile. This one routes to **three** regions; a naive `us-*` guess would have granted four, and the fourth would have been an unused permission.

2. Roles reviewed

Role	Purpose	Statements removed	re-narrowed	Wildcards before	Wildcards after
arn:aws:iam::123456789012:role/BedrockAgentCoreHarnessDefaultServiceRole-fpraf	Invoke model, apply guardrail, invoke gateway, write logs	7	5	6	6 (all justified, §4)
arn:aws:iam::123456789012:role/BedrockAgentCoreGatewayDefaultServiceRole-1789761291328	Retrieve from the knowledge base	2	3	2	0
arn:aws:iam::123456789012:role/BedrockExecutionRoleForKnowledgeBase-jcpa3	Read S3, invoke the embedding model	1 action	1	2	2 (both justified, §4)

Totals: 9 statements removed, 9 narrowed or split, 1 action stripped, 28 individual permissions withdrawn, 8 wildcards surviving with individual justification.

A finding about where the permissions actually live

The project brief implies the over-permission is in inline policies. **It is not.** The console-generated AgentCore roles put almost everything in **customer-managed attached policies**:

Role	Inline	Attached	Where the wildcards were
Harness	1	2	All 6 in the attached ...HarnessExecutionPolicy_le77v
Gateway	0	2	Both in attached policies
Knowledge Base	0	3	Both in attached policies

This matters operationally: narrowing them requires `CreatePolicyVersion(SetAsDefault=True)`, not `PutRolePolicy`. A hardening script that only handles inline policies runs successfully, reports changes, and **leaves every real over-permission in place**. The first version of this script did exactly that.

3. Changes applied

3.1 Harness execution role

NARROWED — `BedrockModelInvocation` (*the headline change*)

Before:

```
"Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
"Resource": [
  "arn:aws:bedrock:*:foundation-model/*",
  "arn:aws:bedrock:us-east-1:118924230273:*"
]
```

After:

```
"Resource": [
  "arn:aws:bedrock:us-east-1:118924230273:inference-profile/us.amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-1:foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-2:foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-west-2:foundation-model/amazon.nova-2-lite-v1:0"
]
```

Rationale. The second ARN in the original — `arn:aws:bedrock:us-east-1:118924230273:*` — is **every Bedrock resource in the account**: every model, guardrail, knowledge base and inference profile, including resources that do not exist yet. The harness invokes exactly one inference profile. Granting `foundation-model/*` would additionally let a compromised role invoke every model in the account, including far more capable and more expensive ones.

NARROWED — `ApplyNorthstarGuardrail`

Scoped to one guardrail and its cross-Region profile across three regions:

```
arn:aws:bedrock:us-east-1:118924230273:guardrail/6yxbopugn44t
arn:aws:bedrock:us-east-1:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-west-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

Rationale, and a finding. `ApplyGuardrail` on `*` would let the role apply **any** guardrail in the account — including a permissive one, which defeats the control it is supposed to enforce.

The four ARNs rather than one is not padding. A STANDARD-tier guardrail runs through cross-Region inference, so authorisation covers the guardrail *and the profile* — and **the profile ARN is regional, following request routing**. Granting only `us-east-1` produced an intermittent failure when a call routed to `us-east-2`:

```
not authorized to perform: bedrock:ApplyGuardrail on resource:
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

The regions are read from the model's inference profile — the guardrail is applied wherever inference runs, so those are exactly the regions needed. Still enumerated; no wildcard. The project brief's sample policy grants only the guardrail ARN and would fail on any STANDARD-tier guardrail.

NARROWED — CloudWatch Logs statements (×3)

logs:CreateLogGroup, CreateLogStream, PutLogEvents, DescribeLogStreams and DescribeLogGroups scoped to this system's log groups.

Rationale. logs:DescribeLogGroups on log-group:* let the agent's own role enumerate every log group in the account. In a broader variant it would permit deleting the logs of its own activity — the classic anti-forensics move.

REMOVED — 7 statements, 28 permissions, all for features that are off

Statement	Actions	Why removed
AgentCoreBrowserDefault	7	Browser tool is off on the harness
AgentCoreCodeInterpreterDefault	5	Code Interpreter is off
AgentCoreMemory	5	Memory is disabled — no events to read or write
EFSClientAccess	2	No EFS mount configured
EFSDescribe	2	As above
S3FilesClientAccess	3	No S3 file-system mount configured
S3FilesDescribe	2	As above

Rationale. A permission with no caller is pure attack surface. These are exactly what a compromised role reaches for: StartBrowserSession gives outbound web access under the agent's identity; InvokeCodeInterpreter gives arbitrary code execution; the file-system mounts give persistent read/write storage. None is needed to answer a policy question, and each was granted by default.

3.2 Gateway service role — 2 wildcards to 0

REMOVED — AllowBedrockAgenticRetrieveFromKnowledgeBase, AllowBedrockRerankForKnowledgeBase

Both granted bedrock:AgenticRetrieveStream / bedrock:Rerank on "*".

Rationale. The gateway target is configured for **Standard retrieval**, not Agentic retrieval. Neither action has a caller in this deployment, and both were granted on every resource. Removing them took this role to **zero wildcards**.

NARROWED — AllowBedrockInvokeModelForKnowledgeBase

From foundation-model/* plus inference-profile/*, provisioned-model/* and custom-model/* — to the Titan embedding ARN only.

Rationale. The gateway touches models solely for retrieval-side embedding. Granting it the generation model would let a compromised retrieval path run inference the design never intended. This is why the script resolves targets **per role** rather than globally: “the model” means different things to the harness and to the gateway.

NARROWED — AllowBedrockApplyGuardrailForKnowledgeBase

From guardrail/* to the specific guardrail.

3.3 Knowledge base service role

Already well scoped by the console — bedrock:InvokeModel on the exact Titan ARN, s3:ListBucket and s3:GetObject on the exact bucket. One change needed.

ACTION-REMOVED — aws-marketplace:Unsubscribe

Before: ["aws-marketplace:Subscribe", "ViewSubscriptions", "Unsubscribe"] on "*" **After:** ["aws-marketplace:Subscribe", "ViewSubscriptions"] on ""

Rationale. Unsubscribe on * lets this role **cancel foundation-model subscriptions for the entire account** — an availability attack against every AI workload in the account, not just this one. The knowledge base role needs to read and create subscriptions in order to use a model; it never needs to revoke one.

This statement also explains an earlier failure. The *harness* role lacked marketplace permissions entirely, which is why every Anthropic model failed with aws-marketplace:Subscribe denied — while this role, on the same account, held all three including the dangerous one. An inconsistency worth flagging on its own.

4. Surviving wildcards — individual justification

The rubric requires that no wildcard remain unless explicitly justified. Eight remain. **Every one is on an action for which AWS does not support resource-level permissions** — the wildcard is not a shortcut, it is the only valid form.

Statement	Actions	Justification
CloudWatchMetricsPublish (har-ness)	cloudwatch:PutMetricData	No resource-level permissions supported. Constraining only by a cloudwatch:namespace condition key; recommended as a follow-up.
CloudWatchWritePermissionStatement (KB)	cloudwatch:PutMetricData	As above.
XRayTracingAccess	xray:PutTraceSegments, PutTelemetryRecords, GetSamplingRules, GetSamplingTargets	X-Ray supports no resource-level permissions for these. Write-only telemetry; no read of other services' traces.
EcrPublicTokenAccess	ecr-public:GetAuthorizationToken	Token-issuing call with no resource to name. Required to pull the harness runtime image.
EcrManagedImageToken	ecr:GetAuthorizationToken	As above. The paired EcrManagedImagePull is scoped, to repository/harness-.*.
StsForEcrPublicPull	sts:GetServiceBearerToken	Bearer-token issuance for the ECR pull; no resource.
BedrockMantleCallWithBearerToken	bedrock-mantle:CallWithBearerToken	Bearer-token call with no resource to name.
MarketplaceOperationsFromBedrockFor3pModels	aws-marketplace:Subscribe, ViewSubscriptions	Account-scoped by nature. Unsubscribe was removed — the dangerous member of the set.

Four of these (ecr-public, ecr, sts:GetServiceBearerToken, bedrock-mantle) are the bootstrap that starts the harness runtime. Removing them would stop the agent running at all — which the verification in §5 would have caught.

5. Verification — and why it was built in

Over-tightening an agent's role produces a failure mode worse than leaving it loose: **the agent keeps answering, but without retrieval**. Answers look plausible, are entirely ungrounded, and nothing in the response indicates it.

So `--apply` is paired with `--verify`, which after IAM propagation runs one live invocation and checks four things: no error, the guardrail did not block a benign question, the Retrieve tool actually fired, and chunks came back non-zero. **On failure it automatically restores every original policy** — inline and managed — replaying from `evidence/iam/before/`.

Result:

```
[18:02:14] OK applied ...HarnessExecutionPolicy_le77v [managed]
[18:02:14] OK applied ...GatewayKBAccessProd_7B6CA2 [managed]
[18:02:14] OK applied ...S3PolicyForKnowledgeBase_pcpa3 [managed]
[18:02:14] ==> waiting 15s for IAM propagation before verifying
[18:02:39] OK harness still answers with retrieval after narrowing
```

That single line confirms three things at once: the guardrail is attached and `ApplyGuardrail` resolves across regions, retrieval still functions through the gateway, and nothing load-bearing was removed. No rollback was needed.

Rollback remains available. All eight original policy documents are preserved, and `--restore` replays them — including managed policies, via a new default version. The 5-version IAM cap is handled by pruning the oldest non-default version first.

6. What an attacker gains — before versus after

With the original console-generated roles, a principal able to assume the harness execution role could:

- Invoke **any** Bedrock model in the account, including more capable and costlier ones — and any guardrail, knowledge base or inference profile, present or future, via `arn:aws:bedrock:us-east-1:118924230273:*`
- Apply **any** guardrail, including a permissive one, neutralising the control
- Start **browser sessions** — outbound web access under the agent's identity
- **Execute arbitrary code** via the Code Interpreter
- Mount **EFS and S3 file systems** — persistent read/write storage
- Read and write **agent memory**
- **Enumerate every log group** in the account

And a principal holding the knowledge base role could **cancel foundation-model subscriptions account-wide**, disabling every AI workload in the account.

After narrowing, the same principal can:

- Invoke **one** inference profile and the three regional model ARNs it routes to
- Apply **one** guardrail, via its own cross-Region profile
- Invoke **one** gateway
- Write to **this system's** log groups
- Publish metrics and traces (no resource-level permissions exist)
- Pull the harness runtime image

The gateway role can read and retrieve from **one** knowledge base and embed with **one** model. The knowledge base role can read **one** bucket, embed with **one** model, and can no longer revoke anything.

Blast radius for a compromise of the agent's identity goes from “the account's entire AI estate, plus code execution and outbound network” to “read 30 internal documents.” That is what least privilege actually bought here — not a tidier policy document.

7. Remaining recommendations

1. Add a **cloudwatch:namespace condition** to both `PutMetricData` statements, restricting them to `NorthstarAssist/Security`. This is the only surviving wildcard that can be meaningfully constrained.
2. Add a **permissions boundary** to all three service roles, so future console operations cannot re-widen them. These roles were generated permissive by default and a console edit could regenerate them that way.

3. **Alert on PutRolePolicy / CreatePolicyVersion / AttachRolePolicy** against these three roles via CloudTrail. Narrowing is only durable if re-widening is visible.
 4. **Populate runtimeUserId** so invocation logs attribute activity to an employee rather than to the shared execution role – see the monitoring plan §1.5.
 5. **Create a break-glass decommissioning role** with credentials independent of the operating ones. During this assessment a credential revocation left live infrastructure running and unreachable; the containment path should not share fate with the operating path.
-

Evidence

Artefact	Location
Original policy documents (8)	iam/before/
Narrowed policy documents (9)	iam/after/
Unified diffs, per policy	iam/after/*.diff
Machine-readable change log	evidence/iam/iam_changes.json
Full before/after capture	evidence/iam/iam_before.json, iam_after.json
Generator output	evidence/iam/generated-change-log.md
Narrowing implementation	scripts/20_harden_iam.py